Reusable Delivery

A practical guide for AI-assisted work

Turn a new request or work already underway into a story someone else can pick up. A story records the outcome, what success looks like, the next action and the evidence behind each result.

Choose	Plan	Verify	Hand off
New work or existing work	Story, outcome and small tasks	Checks and review evidence	Links, owner and next action

What your team gains

A clear place to look	Keep the story, decisions and evidence together so the next person can find the current answer.
A clear next action	Give each task one executor and a result they can check before passing it on.
A traceable handoff	See what was checked, what passed and what still needs a decision.

Start with one link. Open the Reusable Delivery guide. The guide will direct you to the right template.

Start in one chat

Paste this prompt into the AI chat that holds the work, or start a new chat and add a short brief.

```
Use https://github.com/aispanda/Reusable-Delivery/blob/main/DELIVERY.md
for the work in this chat.
```

Choose the route

New work

You have an outcome or brief. Define what success looks like and create the first story.

Existing work

You already have a story, decisions, code or test evidence. Move the work forward with that history intact.



1. **Provide the starting point**

   Say who needs the outcome and what should change. Share the brief or authorized links to the original story, decisions and evidence.

2. **Confirm the destination and scope**

   Give the target GitHub repo and folder if you know them. Say whether the task is to draft, migrate or carry on with authorized delivery work.

3. **Use one guide version**

   The selected guide contains the template and execution rules. Record its exact GitHub commit so the next executor can use the same version.

You can open a route directly when you know which one applies. The chat needs access to the guide and any source material it relies on.

Create a new story

Use the template to write the GitHub story. Say who needs the change, why it matters and how you will check it.

Example story
As a buyer, I want to download my order summary so I can share it with my purchasing team.

Write observable acceptance criteria

AC-01. The download shows the selected order number, date and total for a buyer who can view that order.
AC-02. If access is denied, the request returns no order data.

Give each task one executor

If the story reads or changes an external platform, put **Verify platform access** first in the Execution checklist. Platform discovery, review, build, tests and writes depend on this check. See page 5.

Bounded task	Example executor	Evidence to record
Verify platform access	Assigned executor using the approved tool.	Read-only check result for the intended environment.
Implement the download	AI writer. Example: GPT-6.1 Sol at medium reasoning.	Change revision and writer check results.
Check both criteria	Automation. Repository test command in CI.	Allowed and denied cases, tested revision and result.
Review the change	Independent human reviewer.	Findings, repair decisions and any recheck.
Accept the outcome	Accountable human, when required.	Decision and evidence supporting it.

For the next task, record inputs, output, completion check and when to stop. Include the exact model and reasoning for AI assignments. Record the result and next action.

Move existing work

Begin with the original story and its evidence. Keep agreed criteria, decisions and completed work visible so the next person can continue from the right place.

1. **Inventory the latest sources**

   Read the current story, comments and relevant design, code, decisions and test evidence. Note which version or source supports each important claim.

2. **Preserve the original**

   Before an authorized ClickUp replacement, keep a verbatim copy of the current description in an approved location. Store restricted originals and private values where authorized users can retrieve them.

3. **Compare the draft with the source**

   Keep criterion IDs and wording. Record redactions or conflicts. Carry forward valid results and reopen affected checks. If the story reads or changes an external platform, add **Verify platform access** first in the Execution checklist and keep the existing task IDs.

4. **Verify the handoff**

   Read back the published GitHub story and check readers can open links and retrieve required restricted context. Refresh ClickUp and reconcile edits before an authorized update.

A small source map can fit inside the story

Original	Where it goes	What to record
Agreed criteria	GitHub story	Preserved IDs and wording.
Past test result	Evidence link and tested revision	Scope that result actually covered.
Client-specific values	Approved restricted storage	Who can retrieve the source and how.

If ClickUp writeback or source access is pending, record the gap and keep the original source authoritative. Link a separate story for new scope unless an extension is authorized.

Check the work before handoff

Plan checks before building. The story coordinator chooses checks and reviewers based on the criteria, risk and repository rules.

1 **Verify platform access when the story needs it**

Use the approved connected tool (MCP) or command line (CLI). Run a minimal read-only check for each tool against the intended account and environment. Use browser sign-in only to authenticate a tool.
For Salesforce, verify Headless 360 MCP and Salesforce CLI against the intended org. Start in MCP. Discover, describe, then run `dispatch_readonly`.
In the CLI, run `sf org list --json` and a read-only API check against the intended alias. Use MCP first for org work and the verified CLI as needed. Writes require existing authorization.

Record access results. Log the tools, environment, checks, results and time. Keep private details in restricted storage. If a connection fails, record the redacted error and pause dependent tasks until a recheck passes. Recheck after an executor environment or authorization change.

2 **Run repeatable checks**

Use the same code check locally and in CI. Check success and failure paths. For documentation, verify sources, criteria and links.

3 **Have a fresh reviewer look for gaps**

For material changes, assign a fresh human or approved AI reviewer after writer checks. Fix confirmed issues and rerun affected checks.

4 **Record the human decision**

When acceptance or release approval is required, the accountable person records the decision and evidence.

Record the check, tested revision, environment, result and evidence link. Mark unrun checks as NOT RUN.

What this looked like in the shipment pilot

The same check ran locally and in GitHub Actions. It passed 22 offline tests on each of two Python versions using simulated shipments. A separate CI run caught a deliberately failing test.

Code quality checklist · Pilot evidence · AI Spanda QA report

Hand off work so it can continue

Once the GitHub story is published and checked, connect it to ClickUp and leave a clear next action.

GitHub holds the delivery record

Keep the story, acceptance criteria, task checklist, decisions and evidence together. Link detailed designs or debugging notes where they help.

ClickUp tracks the work

Keep the title, optional story sentence, GitHub folder link and story link. Use ClickUp fields for ownership, dates and status.

Leave the next person three things

1. **The current story and revision**

   Share the story link and the exact guide or artifact revision the person should use.

2. **The result and its limits**

   Show what passed and what is still open. Check what you publish for sensitive details. Use placeholders in public GitHub and keep private originals and their mapping in approved restricted storage.

3. **The next eligible task**

   Name the executor, what they need and what they should deliver. Record the condition for resuming a blocked task.

If you are handing off reusable code, include setup, environment inputs, the test command and verified installation evidence for the supported setup. Keep credentials in the approved environment.

Open Reusable Delivery on GitHub · Guide revision used for this PDF